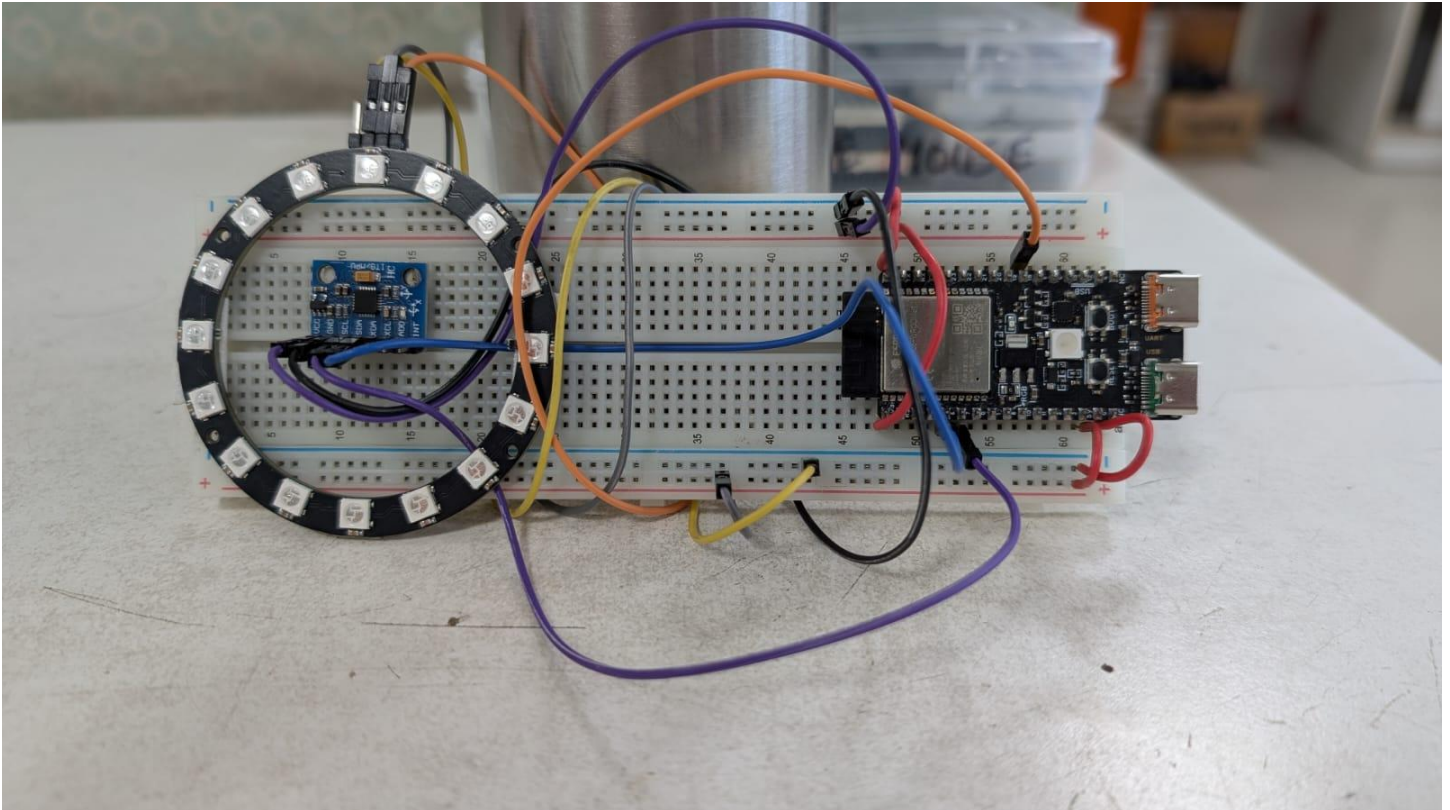


Rolling Marble Simulation Playbook

I. Challenge

Design and execute an embedded Hardware-in-the-Loop (HIL) physics engine that models the dynamic kinematics of a virtual sphere rolling along a circular path. The system must translate continuous spatial data from an inertial sensor into dynamic physical properties—such as inertia, momentum, friction, and mass—and map the resulting high-precision floating-point coordinates onto a discrete 16-LED addressable display ring with zero latency at a steady 50 Hz execution loop.



II. Guide

A. Concept

To implement a responsive, realistic physics simulation on a microcontroller, we apply these key trigonometric and classical mechanics concepts:

- **1. Downhill Incline Resolver:** Micro-electromechanical system (MEMS) accelerometers measure raw gravitational vectors along spatial axes. To isolate the true direction that gravity pulls a physical object downwards, the system relies on the two-argument arctangent function:

$$\theta_{\text{target}} = \text{atan2}(x_{\text{Angle}}, y_{\text{Angle}}) \times \frac{180}{\pi}$$

This evaluates the quadrant signs to map the absolute tilt target from 0° to 360°.

- Learn more: [The Mathematics of the Atan2 Function](#)

- **Modular Angular Distance & Wrap-Around:** Direct subtraction ($\theta_{\text{target}} - \theta_{\text{marble}}$) breaks down across the $0^\circ/360^\circ$ boundary, causing false rotational directions. The physics engine constrains angular error to $[-180^\circ, 180^\circ]$ to ensure the simulated marble always takes the shortest physical path around the circular track.

- *Learn More:* [Stack Overflow – Finding the Shortest Distance Between Two Angles](#)

- **3. Semi-Implicit Euler Numerical Integration:** Instead of complex continuous equations, the physics engine updates the particle across tiny discrete slices of time ($\Delta t = 20$ ms). Restoring torque yields acceleration, which modifies the current velocity before being compressed by a damping factor (μ) representing surface friction:

$$v_t = (v_{t-1} + a) \times \mu$$

$$x_t = x_{t-1} + v_t$$

- *Learn more:* [Numerical Integration](#)

- **4. Continuous-to-Discrete Remapping:** The physics core operates entirely with high-precision floating-point variables to maintain momentum accuracy. To display this on hardware, the continuous angle must be scaled, rounded to the nearest integer index, and wrapped using a modulo operation to light the correct pixel:

$$\text{Index}_{\text{LED}} = \left\lfloor \text{round} \left(\frac{\theta_{\text{marble}}}{360/N_{\text{LEDs}}} \right) \right\rfloor \pmod{N_{\text{LEDs}}}$$

- *Learn more:* [Quantization Error Explained](#)

Real-World Example: Advanced stability control systems in aerospace rocketry, smartphone screen orientation adjustments, and active anti-sway suspension structures in high-performance automotive platforms. They sample real-time tilt vectors to shift virtual or mechanical elements against gravity.

B. Components

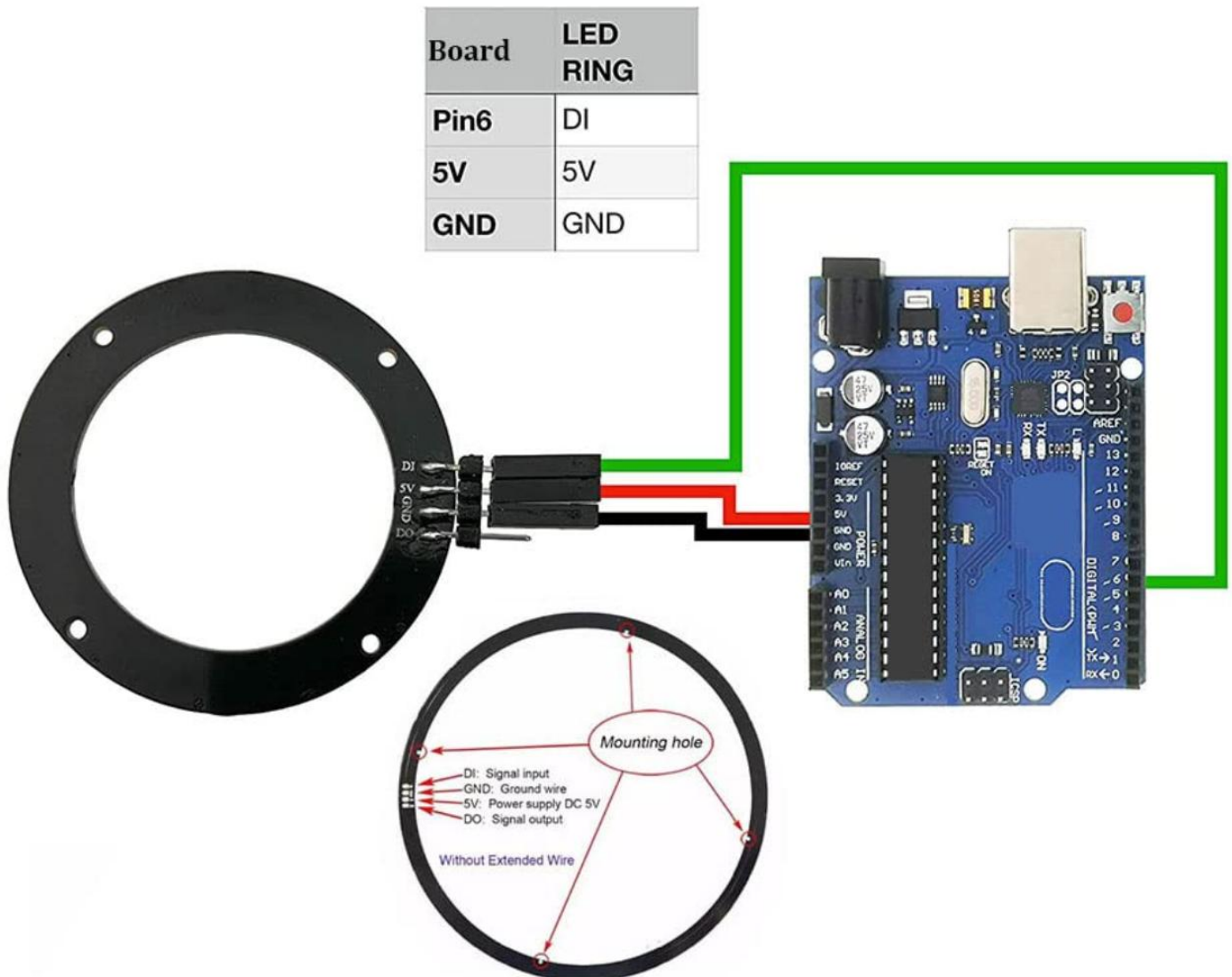
- **Microcontroller:** WeAct Studio ESP32-C6 Dev Board.
- **Motion Sensor:** MPU6050 6-DoF Inertial Measurement Unit.
- **Visualizer Array:** 16-element WS2812B NeoPixel RGB LED Ring.
- **Interconnect Base:** Solderless breadboard and jumper hookup wires.

C. Connections

The system uses an I^2C protocol channel to extract the sensor readings and a high-speed one-wire pulse control channel to update the lighting ring:

Source Device	Pin Label	Target MCU Pin (ESP32-C6)	Operational Functional Description
MPU6050 IMU	VCC	3.3V	Logic Power Supply
	GND	Gnd	Common Ground Reference
	SDA	GPIO 6	Hardware I^2C Serial Data Communication
	SCL	GPIO 7	Hardware I^2C Serial Clock Line
NeoPixel Ring	5V	V5 / VIN	High-Current Input (Direct USB 5V Rail)
	GND	Gnd	Common Ground Reference
	DI	GPIO 18	High-Speed Digital Pixel Control Input
	DO	Unconnected	Open Daisy-Chain Output Pass-through

How to connect RGB LED Ring Module is explained as shown by this diagram:



D. Code

This program sets up the hardware lines, calibrates the gyroscope base offset, and runs the integration equations at a consistent 50 Hz step interval:

```
1. #include <Wire.h>
2. #include <MPU6050_tockn.h>
3. #include <Adafruit_NeoPixel.h>
4.
5. #define LED_PIN    18
6. #define NUM_LEDS   16
7. #define I2C_SDA     6
8. #define I2C_SCL     7
9.
10. MPU6050 mpu6050(Wire);
11. Adafruit_NeoPixel ring(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
12.
13. // --- PHYSICS ENGINE VARIABLES ---
14. float marblePos = 0.0;    // Current position of the marble in degrees (0.0 to 360.0)
15. float marbleVelocity = 0.0; // Current angular velocity
```

```

16. const float FRICTION = 0.92; // Damping factor (0.99 = ice/no friction, 0.80 = heavy mud)
17. const float GRAVITY_MULT = 0.02; // How responsive the marble is to gravity/weight
18.
19. void setup() {
20.   Serial.begin(115200);
21.   Wire.begin(I2C_SDA, I2C_SCL);
22.
23.   ring.begin();
24.   ring.setBrightness(40);
25.   ring.show();
26.
27.   Serial.println("Place flat for IMU calibration...");
28.   mpu6050.begin();
29.   mpu6050.calcGyroOffsets(true);
30.   Serial.println("Physics Engine Ready!");
31. }
32.
33. void loop() {
34.   mpu6050.update();
35.   float xAngle = mpu6050.getAngleX();
36.   float yAngle = mpu6050.getAngleY();
37.
38.   // 1. Calculate overall tilt steepness (Magnitude)
39.   float tiltMagnitude = sqrt((xAngle * xAngle) + (yAngle * yAngle));
40.
41.   // 2. Only update physics if the board is actually tilted
42.   if (tiltMagnitude > 2.0) {
43.     // Find the downhill target angle where gravity pulls a marble (opposite of bubble level)
44.     float targetAngle = atan2(xAngle, yAngle) * 180.0 / M_PI;
45.     if (targetAngle < 0) targetAngle += 360.0;
46.
47.     // 3. Calculate shortest angular distance to target (handles 0/360 wrap-around)
48.     float angularError = targetAngle - marblePos;
49.     if (angularError > 180.0) angularError -= 360.0;
50.     if (angularError < -180.0) angularError += 360.0;
51.
52.     // 4. Euler Integration: Accel -> Velocity -> Position
53.     float acceleration = angularError * tiltMagnitude * GRAVITY_MULT;
54.
55.     marbleVelocity += acceleration;
56.     marbleVelocity *= FRICTION; // Apply simulated surface resistance
57.     marblePos += marbleVelocity;
58.
59.     // Keep marble position strictly within 0-360 degrees
60.     if (marblePos >= 360.0) marblePos -= 360.0;
61.     if (marblePos < 0.0) marblePos += 360.0;
62.   } else {
63.     // If the board is perfectly flat, slowly bleed off any remaining velocity
64.     marbleVelocity *= 0.5;
65.     marblePos += marbleVelocity;
66.   }
67.
68.   // 5. Map the float position angle to the nearest integer LED index
69.   int currentLED = ((int)round(marblePos / (360.0 / NUM_LEDS))) % NUM_LEDS;
70.
71.   // 6. Render the Marble
72.   ring.clear();
73.
74.   // Draw the main marble (Bright Blue)
75.   ring.setPixelColor(currentLED, ring.Color(0, 150, 255));
76.
77.   // Draw a dim "tail" or motion blur behind the marble based on velocity direction
78.   int tailedLED = (marbleVelocity > 0) ? (currentLED - 1 + NUM_LEDS) % NUM_LEDS : (currentLED + 1) % NUM_LEDS;
79.   if (abs(marbleVelocity) > 2.0) {
80.     ring.setPixelColor(tailedLED, ring.Color(0, 40, 100));
81.   }
82.   ring.show();
83.
84.   delay(20); // Steady 50Hz frame rate for smooth physics integration
85. }
86.

```

III. Play & Share

- **Introduce Elastic Boundary Collisions:** Instead of allowing the marble to spin continuously all the way around the ring, create a virtual brick barrier at LED index 0 and 8. Modify your integration code so that if the marble's position hits a barrier, the position stops instantly, and the velocity multiplies by -0.6 to bounce backwards!
- **Simulate Multi-Body Physics:** Expand your array parameters to introduce a second marble onto the track. Can you write a software check that senses when both indexes share the same discrete space and performs an **elastic momentum swap** so the particles bump away from each other?
- **Design a Dynamic Maze Game:** Use the code to build a game where a player tilts the board to guide the marble into a designated, blinking "target gate" LED within a specific time limit while avoiding dynamic, red "trap" LEDs that reset the score.